Three samples, no wasted cycle

Separate controller state, sample position and accumulated data, then follow adjacent three-sample groups without gaps.

Kapil Tripathi | Verilog & digital design | 12 pages

CONTENTS / click a row to jump

Start edge, sample numbering and a 101 trace	2-3
Failed attempts: loop, count and gap-cycle mistakes	4-8
Combinational defaults and corrected RTL	9-10
All eight groups and local verification	11
Revision checklist and sources	12

KEEP THIS IDEA

At the third edge, the stored count contains the first two samples. Include the current input before deciding the result.

Tracker entries: 90

Study notes by Kapil Tripathi, based on HDLBits exercises. Original questions, source references and dated verification records are retained in the notes.

ALL NOTES + ENTRY INDEX HDLBits problem collection

1. Translate the problem into an exact clock sequence

The official problem starts in state A. While **s = 0**, the controller stays in A. The edge that observes **s = 1** moves the controller into state B. Only after it is in B does it examine **w** for the next three rising edges. The result **z** is then visible during the clock cycle following the third sample. B immediately begins the next three-sample group; there is no unused gap cycle.

Phase	What happens on the rising edge	Meaning after the edge
Wait	In A, s = 0	Stay in A; clear the group datapath
Start	In A, s = 1	Enter B; this edge is not w sample 1
Sample 1	Old ticks = 0	Store w1; ticks becomes 1
Sample 2	Old ticks = 1	Store w2; ticks becomes 2
Sample 3	Old ticks = 2	Classify count + w3; reset group counters
Continue	Next rising edge in B	Start sample 1 of the next group; z returns to 0

Two controller states
A waits for s; B keeps sampling. The counters and zreg also store state. This is a two-state controller plus a datapath; the complete circuit has more than two possible stored configurations.

Separate three independent responsibilities

- **state** answers whether sampling has started.
- **ticks** answers how many earlier samples in the current group have already been processed.
- **count** answers how many of those earlier samples were 1.
- **zreg** records the classification result for the just-finished group.

2. The zero-based timing that makes ticks == 2 correct

At the start of a group, **ticks = 0**. This does not mean 'sample zero'. It means zero previous samples have been completed. Because a clocked block reads the old register values, the third sampling edge sees **ticks = 2**.

Sampling edge	Old ticks read by the block	Old count meaning	Current input
First	0	Ones in zero previous samples	w1
Second	1	Ones in w1	w2
Third	2	Ones in w1 and w2	w3

Worked trace: w1w2w3 = 101

Edge	Old ticks	Old count	w	Action	After edge
1	0	0	1	Increment ticks and count	ticks=1, count=1, z=0
2	1	1	0	Increment ticks only	ticks=2, count=1, z=0
3	2	1	1	Test 1 + 1 = 2; reset group	ticks=0, count=0, z=1

On the third edge, count contains the ones in the first two samples. Adding the current w gives the total for all three, which is the value the result test needs.

Invariant
Immediately before each sampling edge in B: ticks equals the number of earlier samples already processed, and count equals the number of 1s among those earlier samples.

3. The earliest for-loop attempt samples one value three times

The supplied screenshot preserves the first structural mistake. A procedural **for** loop inside one **always @(posedge clk)** activation does not create future clock edges. Synthesis unrolls the loop as logic evaluated during the same edge, so all three iterations see the same current **w**.

```
13      localparam s1= 2'b01;
14      localparam s2 = 2'b10 ;
15
16      always @(posedge clk)begin
17          if(reset)begin
18            state<= s0 ;
19              count <= 2'b00;
20          end else begin
21            state<= next_state ;
22              if(state == s1)begin
23                  count<= 2'b00;
24                  for(i=0 ; i<3; i= i+ 1 )begin
25                      if(w)
26                          count<= count + 2'b1;
27                      else
28                          count <= count;
29                  end
30              end
31          end
32      end
33      always @(*)begin
34        next_state = state;
35          case (state)
36              s0: next_state = (s)? s1: s0 ;
37              s1: begin
38                  if(count== 2'b10) begin
39                    next_state = s2;
40                  end else begin
```

In the shown loop, each iteration also executes a nonblocking assignment such as **count <= count + 1**. Every right-hand side reads the same old count. Three scheduled writes do not accumulate to old count + 3; when several nonblocking assignments target the same register in one block, the last executed assignment determines the final scheduled value.

What the code appears to say	What hardware semantics actually do
Loop three times	Replicate/evaluate logic three times in one clock event
Read w three times	Read the same present w value three times
Increment count three times	Schedule old count + 1 repeatedly; no sequential accumulation
Represent three clock cycles	No. Only an FSM/counter can retain progress across edges

4. The improved three-state version is still wrong

Adding ticks tracks separate edges. Two functional errors remain: count increments for zero inputs, and s2 skips an input. The repeated assignments also make those errors harder to see.

```
module top_module (
    input clk,
    input reset,
    input s,
    input w,
    output z
);
    reg [1:0] state, next_state;
    reg [1:0] count;
    integer i;
    localparam s0 = 2'b00;
    localparam s1 = 2'b01;
    localparam s2 = 2'b10;
    reg [1:0] ticks = 2'b00;
    reg zreg = 0;

    always @(posedge clk) begin
        if (reset) begin
            state <= s0;
            count <= 2'b00;
            ticks <= 2'b00;
            zreg <= 1'b0;
        end else begin
            state <= next_state;
            zreg <= 1'b0;

            if (state == s1) begin
                ticks <= ticks + 2'b01;
                count <= count + 2'b01;

                if (ticks == 2'b10) begin
                    count <= 2'b00;
                    ticks <= 2'b00;
                    zreg <= ((count + {1'b0, w}) == 2'b10);
                    count <= 2'b00;
                    ticks <= 2'b00;
                end else begin
                    ticks <= ticks + 2'b01;
                    if (w)
                        count <= count + 2'b01;
                end
            end

            if (state == s2) begin
                count <= 2'b00;
                ticks <= 2'b00;
            end
        end
    end

    always @(*) begin
        next_state = state;
        case (state)
            s0: next_state = s ? s1 : s0;
            s1: begin
                if (ticks == 2'b10)
                    next_state = s2;
                else
                    next_state = s1;
            end
            s2: next_state = s1;
        endcase
    end

    assign z = zreg;
endmodule
```

Bug 1: count is incremented even when w = 0

```
count <= count + 2'b01;      // unconditional
...
if (w)
    count <= count + 2'b01;  // conditional, but same old RHS basis
```

The first assignment makes **count** track sampling edges, not the number of ones. When w = 1, the later assignment does not add a second increment; both assignments calculate old count + 1. When w = 0, the unconditional assignment still increments. Therefore, by the third edge old count tends to be 2 regardless of the first two w values, and the result degenerates into a test dominated by the current w.

Cleanup: duplicate ticks assignments do not increment twice

```
ticks <= ticks + 2'b01;
...
ticks <= ticks + 2'b01;
```

Both right-hand sides read the same pre-edge value. The second scheduled write wins, but its value is still old ticks + 1. This is legal but misleading. Each register should have one clear owner and one intended next value per path.

Bug 3: state s2 inserts an unwanted gap cycle

On the third-sample edge, the old state is still s1, so the datapath classifies the group while the state register schedules s2. During the next edge, old state is s2; the code only clears counters and schedules s1. No w sample is collected on that edge. The problem requires continuous groups, so the extra state creates a dead cycle.

Cleanup: repeated resets hide where the group ends

Resetting count and ticks multiple times in the same branch does not add safety. It makes it harder to see the essential rule: every time the third sample completes a group, both counters reset exactly once, whether the result is true or false.

5. The near-correct bug from the saved Q&A

Q: Why was the version with count + w still wrong?

The classification expression was correct, but the counter resets were nested inside the 'exactly two ones' condition:

```
if (ticks == 2'd2) begin
    if ((count + {1'b0, w}) == 2'd2) begin
        zreg <= 1'b1;
        ticks <= 2'd0;
        count <= 2'd0;
    end
end
```

This confuses two separate decisions. **Window completion** asks whether three samples have finished; if yes, the counters must always restart. **Classification** asks whether exactly two of those samples were 1; it controls only z.

Group	Classification	Required counter action	Buggy action
011, 101, 110	z = 1	Reset	Resets accidentally
000, 001, 010, 100, 111	z = 0	Reset	Does not reset

Failure trace: 111

Before sample 3, ticks = 2 and count = 2. Current w = 1, so count + w = 3 and z correctly stays 0. But the inner condition is false, so ticks remains 2 and count remains 2 (or 3 in a variant that also schedules an increment). Every later edge is misinterpreted as another third sample. The next group never begins cleanly.

Correct nesting
The third-sample condition owns classification and unconditional group reset. The equality test owns only the z value.

```
if (ticks == 2'd2) begin
    zreg <= ((count + {1'b0, w}) == 2'd2);
    ticks <= 2'd0;
    count <= 2'd0;
end
```

6. Saved Q&A: why not count all three first and then test

Q: Why do I need the 2 + current-input approach? Why can the counter not reach 3 and then I check count?

You can define a counter in many ways, but a nonblocking assignment does not update the register immediately inside the current clocked block. On sample 3, **count <= count + w** schedules a new value for the end of the time step. An **if (count == 2)** later in the same block still reads the old count containing only samples 1 and 2.

```
// On the third edge, old count is still the first-two-sample total.
if (w)
    count <= count + 2'd1;

if (count == 2'd2)          // Reads old count, not the scheduled value
    zreg <= 1'b1;
```

For 101, old count is 1 and current w is 1. The assignment schedules count = 2, but the following test still sees count = 1 and would incorrectly output 0. The expression **count + w** explicitly computes the value that the register is about to hold.

Could I wait one more clock?

Yes, but then the result would be evaluated on a fourth edge. That edge is already supposed to be sample 1 of the next group. You would need extra state or parallel bookkeeping to avoid losing that next input, and z would be late relative to the required interface. Computing the total on edge 3 is simpler and matches the problem.

Could ticks count 1, 2, 3 instead?

Yes. The meaning must change consistently from 'completed previous samples' to 'current sample number'. This is the valid one-based form:

```
// Alternative meaning: sample_no is the current sample number (1, 2, or 3).
if (state == A) begin
    sample_no <= 2'd1;
    count <= 2'd0;
end else if (sample_no == 2'd3) begin
    zreg <= ((count + {1'b0, w}) == 2'd2);
    sample_no <= 2'd1;
    count <= 2'd0;
end else begin
    sample_no <= sample_no + 2'd1;
    if (w)
        count <= count + 2'd1;
end
```

Zero-based and one-based counters are both valid. The zero-based definition starts naturally at 0 and makes the invariant particularly clear, so it is usually easier to audit.

7. Why defaults matter in combinational logic, but not in the same way in clocked logic

Q: Why should I give a default only to combinational logic?

A combinational block describes logic with no memory. Every output assigned by that block must receive a value for every possible input/state path. If a path leaves **next_state** unassigned, hardware must remember its previous value, which infers a latch. The line **next_state = state;** at the top is already a default assignment that covers branches that do not override it.

```
always @(*) begin
    next_state = state;          // Default prevents a latch
    case (state)
        A:       next_state = s ? B : A;
        B:       next_state = B;
        default: next_state = A; // Recovery from illegal encoding
    endcase
end
```

The case **default** serves a related but different purpose: it gives illegal or X-like state encodings a defined recovery route. The top assignment prevents incomplete-path storage; the case default improves recovery and simulation clarity.

A clocked block intentionally describes registers. If a register is not assigned on a clock path, it simply holds its old value, which is normal flip-flop behavior. Therefore, a blanket assignment is not required merely to avoid a latch. However, an explicit clocked assignment can still be required by the function. Here, **zreg <= 0** on each non-reset edge makes z a one-cycle pulse unless a completed group overrides it with 1.

Block	Omitted assignment means	Typical use of a default
always @(*)	Unintended storage/latch	Assign every output first, then override
always @(posedge clk)	Flip-flop holds its value	Use only when the intended next value demands it

8. Clean corrected solution

This version keeps the user's successful two-state structure, removes duplicate assignments, makes widths explicit, and documents the datapath invariant.

```
module top_module (
    input  wire clk,
    input  wire reset,   // Synchronous reset
    input  wire s,
    input  wire w,
    output wire z
);
    localparam A = 1'b0;
    localparam B = 1'b1;

    reg state, next_state;
    reg [1:0] ticks;
    reg [1:0] count;
    reg zreg;

    always @(*) begin
        next_state = state;          // Complete combinational assignment
        case (state)
            A:       next_state = s ? B : A;
            B:       next_state = B;
            default: next_state = A; // Illegal-state recovery
        endcase
    end

    always @(posedge clk) begin
        if (reset) begin
            state <= A;
            ticks <= 2'd0;
            count <= 2'd0;
            zreg <= 1'b0;
        end else begin
            state <= next_state;
            zreg <= 1'b0;            // z is a one-cycle result pulse

            if (state == A) begin
                ticks <= 2'd0;
                count <= 2'd0;
            end else if (ticks == 2'd2) begin
                zreg <= ((count + {1'b0, w}) == 2'd2);
                ticks <= 2'd0;       // End every three-sample window
                count <= 2'd0;       // regardless of the result
            end else begin
                ticks <= ticks + 2'd1;
                if (w)
                    count <= count + 2'd1;
            end
        end
    end

    assign z = zreg;
endmodule
```

9. Exhaustive result table

Exactly two ones among three samples means a population count of 2. There are three matching patterns and five non-matching patterns:

w1w2w3	Number of ones	Expected z after sample 3	Counter reset?
000	0	0	Yes
001	1	0	Yes
010	1	0	Yes
011	2	1	Yes
100	1	0	Yes
101	2	1	Yes
110	2	1	Yes
111	3	0	Yes

Local verification record

- The corrected implementation passed all eight possible windows.
- Additional 101, 111, and 110 groups were applied back-to-back to verify continuous grouping.
- The preserved reset-only-on-match design diverged as expected, proving the testbench exercises the documented failure.

```
PASS: q3fsm correct design passed all exhaustive and back-to-back windows;
buggy reset placement diverged as expected.
```

10. Revision checklist

- Write the clock sequence before writing RTL: start edge, sample edges, output cycle, and restart edge.
- Give each counter one precise sentence definition. If the definition is fuzzy, off-by-one bugs follow.
- Remember that all right-hand sides in a clocked block read the pre-edge state.
- Use count + current_bit when the current bit has not yet entered the registered count.
- Reset group-tracking counters because the group ended, not because a particular result occurred.
- Avoid multiple nonblocking assignments to one register on the same path unless priority is deliberate and obvious.
- Do not use a procedural for loop to represent future clock cycles.
- Defaults in combinational logic prevent latches; missing clocked assignments normally mean hold.
- Use the minimum two control states here; ticks and count belong to the datapath.

Sources and evidence

HDLBits Exams/2014 q3fsm - official problem contract and module declaration.

User-supplied code and screenshot: early loop attempt plus improved and corrected versions, supplied 2026-09-03.

User's saved Edge conversation 'Verilog FSM debugging' - Q&A about reset placement, zero-based ticks, count + w, and one-based alternatives, reviewed locally 2026-09-03.

Local verification: internal/Verification/day6_q3fsm_selfcheck.sv.